

Computer Science Department
COMP4382 - Final Project Guidelines

Land Registration Management Information System
(LRMIS)

Project Overview

The Land Registration Management Information System (LRMIS) is an advanced, secure, geo-enabled platform for managing land registration services, property ownership records, parcel-related applications, cadastral information, field survey tasks, registrar review, objections, and official certificate issuance.

Unlike a simple document submission system, LRMIS enforces a full registration workflow, validates ownership and parcel data, supports geospatial parcel management, tracks every action through immutable audit logs, and provides dashboards for managers and decision makers.

The system enables the Land Authority / Municipality to:

- Receive land registration applications from citizens, lawyers, surveyors, or authorized representatives.
- Manage parcels using GeoJSON boundaries and spatial indexing.
- Validate ownership documents, parcel numbers, applicant information, and attached evidence.
- Route applications through a strict workflow: submitted -> pre_checked -> survey_required -> surveyed -> legal_review -> approved -> certificate_issued -> closed.
- Assign field survey tasks based on zone, surveyor availability, skills, and workload.
- Track objections, disputes, missing documents, and registrar decisions.
- Generate official land registration certificates and application status reports.
- Provide operational dashboards, spatial maps, application analytics, and compliance-ready reports.

Goal: Improve land registration transparency, efficiency, accuracy, and accountability through a digital, workflow-driven, geospatial Management Information System.

Students will develop LRMIS using FastAPI (Python) + MongoDB (PyMongo), with a frontend of their choice: React, Vue.js, Angular, or Streamlit. The system consists of four main modules. Each of the three students will be responsible for one major module, and the fourth module will be implemented by the group.

Key Features

Land Application Management

- Submit new land registration applications.
- Validate required applicant, parcel, ownership, and document fields.
- Support application types: first registration, ownership transfer, parcel subdivision, parcel merge, boundary correction, certificate request.
- Enforce workflow/state transitions.
- Track missing documents, objections, and registrar notes.
- Store attachments metadata and document verification state.

Applicant Portal

- Create applicant profiles.
- Allow citizens, lawyers, surveyors, and representatives to submit applications.
- Track application status and timeline.
- Upload additional required documents.
- Receive notifications using email/SMS stubs.
- Submit objections or responses when requested.

Surveyors, Registrar, and Assignment

- Manage surveyors and land registration officers.
- Assign field survey tasks based on parcel zone, workload, availability, and skill.
- Record field visit milestones.
- Upload survey evidence and field reports.
- Support registrar review and approval decisions.

Geospatial Map and Analytics

- Display parcels, pending applications, disputed parcels, and survey tasks on a map.
- Use GeoJSON and MongoDB 2dsphere indexes.

- Generate heat maps of pending applications and disputed areas.
- Provide analytics on processing time, backlog, approval rates, survey workload, and application types.

Project Modules and Responsibilities

1. Land Application Management Module - Student 1

Responsibilities:

- Implement robust CRUD operations for land registration applications.
- Support validation, pagination, filtering, and sorting.
- Implement idempotency for application creation to avoid duplicate submissions.
- Implement a strict workflow/state machine.
- Enforce mandatory fields per transition.
- Support manual registrar decisions.
- Store parcel information using GeoJSON.
- Support parcel number, block number, basin number, zone, owner references, and application type.
- Track attached documents and their verification status.
- Store internal notes and registrar remarks.
- Record all important events in performance logs.

submitted -> pre_checked -> survey_required -> surveyed -> legal_review -> approved -> certificate_issued -> closed

Alternative states: rejected, on_hold, missing_documents, under_objection

Rules / Required Items:

- An application cannot move to pre_checked unless applicant and parcel information are complete.
- An application cannot move to survey_required unless the parcel location is valid.
- An application cannot move to surveyed unless a survey report exists.
- An application cannot move to legal_review unless ownership documents are uploaded.
- An application cannot move to approved unless legal review is completed.
- A certificate cannot be issued unless the application is approved.
- Rejected applications must include a rejection reason.
- Applications with objections must move to under_objection.

Key API Endpoints:

Endpoint	Description
POST /applications/	Create a new land registration application. Supports idempotency key.
GET /applications/{application_id}	Retrieve full application details, including workflow state, parcel data, attachments, survey status, objections, and certificate state.
GET /applications/	List applications with filters, pagination, and sorting.
PATCH /applications/{application_id}/transition	Move application from one workflow state to another.
POST /applications/{application_id}/hold	Place application on hold with reason.
POST /applications/{application_id}/reject	Reject application with mandatory legal/administrative reason.
POST /applications/{application_id}/certificate	Generate certificate metadata after approval.

MongoDB Collections / Operators:

- land_applications
- parcels
- performance_logs
- certificates

2. Applicant Portal and Profiles Module - Student 2

Responsibilities:

- Design applicant profiles.
- Support applicant types: citizen, lawyer, company, surveyor, authorized representative.
- Support verification states: unverified, verified, suspended.
- Allow applicants to submit land-related applications.
- Allow applicants to upload required documents.
- Allow applicants to add comments or responses.
- Allow applicants to submit objections.
- Allow applicants to track application status.
- Show application timeline.
- Stub notifications using email/SMS fields.

Rules / Required Items:

- Full name
- National ID / registration number
- Contact details
- Address
- Applicant type
- Verification state
- Preferred language
- Notification preferences
- Linked applications
- Privacy settings

Key API Endpoints:

Endpoint	Description
POST /applicants/	Create applicant profile.
GET /applicants/{applicant_id}	Retrieve applicant profile with restricted fields.
GET /applicants/{applicant_id}/applications	Show all applications submitted by the applicant.
POST /applications/{application_id}/documents	Add document metadata or upload supporting document.
POST /applications/{application_id}/comments	Add applicant comment or response.
POST /applications/{application_id}/objections	Submit an objection.
GET /applications/{application_id}/timeline	View status timeline and history.

MongoDB Collections / Operators:

- applicants
- application_documents
- objections
- performance_logs

3. Surveyors, Registrar, and Assignment Module - Student 3

Responsibilities:

- Create and manage surveyors and registrar staff.
- Define coverage zones for surveyors.
- Define skills and specialization.
- Define schedules and availability.
- Implement automatic survey assignment.
- Support manual reassignment.
- Track field survey milestones.
- Upload survey report metadata.

- Allow registrar staff to review survey results.
- Add internal decision notes.
- Implement basic access control for staff-only endpoints.

Assignment policy: zone match + surveyor availability + workload balancing + skill match + priority + existing assigned tasks.

Survey milestones: assigned -> visit_scheduled -> arrived_on_site -> survey_started -> survey_completed -> report_uploaded -> registrar_reviewed

Key API Endpoints:

Endpoint	Description
POST /staff/	Create surveyor or registrar staff account.
GET /staff/{staff_id}	Retrieve staff profile, workload, and performance summary.
POST /applications/{application_id}/auto-assign-surveyor	Automatically assign surveyor based on policy.
PATCH /applications/{application_id}/survey-milestone	Add survey milestone.
POST /applications/{application_id}/survey-report	Upload or register survey report metadata.
PATCH /applications/{application_id}/registrar-review	Registrar review decision.

MongoDB Collections / Operators:

- staff_members
- survey_tasks
- survey_reports
- performance_logs

4. Data Analysis, Map, and Visualization Module - Group Module

Responsibilities:

- Build dashboards for application backlog, approval rates, survey workload, and certificate issuance.
- Build a live map showing land parcels and applications.
- Provide spatial filters by zone, parcel status, application type, and dispute state.
- Generate reports for management.
- Use MongoDB aggregation pipelines.
- Use MongoDB geospatial queries.
- Support export to CSV or PDF.
- Cache heavy analytics queries when needed.

Rules / Required Items:

- Total applications
- Applications by status
- Applications by type
- Pending applications
- Approved applications
- Rejected applications
- Applications under objection
- Average processing time
- Surveyor workload
- Registrar workload
- Certificates issued per month
- Hotspot zones with high number of applications
- Delayed applications

Key API Endpoints:

Endpoint	Description
GET /analytics/kpis	Return main KPIs.
GET /analytics/applications-by-status	Return number of applications grouped by status.
GET /analytics/applications-by-zone	Return applications grouped by zone.
GET /analytics/processing-time	Return average processing time by application type.
GET /analytics/surveyors	Return surveyor productivity and workload analytics.
GET /analytics/registrars	Return registrar review workload.
GET /analytics/geofeeds/parcels	Return parcel GeoJSON feed.
GET /analytics/geofeeds/pending-heatmap	Return heat-map GeoJSON for pending applications.

MongoDB Collections / Operators:

- \$lookup
- \$facet
- \$bucketAuto
- \$geoNear
- \$group
- \$match
- \$sort
- \$project
- \$unwind

Project Deliverables

1. Working FastAPI backend.
2. MongoDB integration using PyMongo.
3. Workflow transitions and validation rules.
4. Automatic surveyor assignment.
5. Applicant portal.
6. Staff console.
7. Surveyor interface.
8. Registrar review interface.
9. Analytics dashboard.
10. Interactive map using OpenStreetMap + Leaflet.
11. GitHub repository.
12. README file including setup instructions, packages, environment variables, MongoDB indexes, sample users, and run instructions.
13. OpenAPI/Postman documentation.
14. Example API requests.
15. Final presentation including system demo, code walkthrough, database design, workflow explanation, design decisions, challenges, and solutions.

Evaluation Criteria

16. Coverage of the required functionality.
17. Correct implementation of FastAPI and MongoDB.
18. Proper use of Pydantic validation.
19. Correct workflow/state machine implementation.
20. Correct use of MongoDB collections and indexes.
21. Proper use of GeoJSON and geospatial queries.
22. Quality of API design.
23. Quality of frontend interfaces.
24. Quality of analytics and dashboards.
25. Code organization and readability.
26. GitHub repository quality.

- 27. Final presentation and demo.
- 28. Ability to explain and modify the code during discussion.

Submission

29. Due date: 23-6-2026

- 30. Submit the project as a GitHub repository link.
- 31. Submit a .zip copy of the project.
- 32. No late submissions or extensions will be accepted.
- 33. Students may be asked to design and apply code changes during the discussion.
- 34. Every student must be able to explain their own module and the group module.

Sample MongoDB Schema Design and Sample Data

Land Applications Collection - land_applications

```
{
  "_id": {
    "$oid": "675100000000000000000001"
  },
  "application_id": "LRMIS-2026-0001",
  "application_type": "ownership_transfer",
  "status": "pre_checked",
  "priority": "normal",
  "applicant_ref": {
    "applicant_id": {
      "$oid": "6751000000000000000000101"
    },
    "applicant_type": "citizen",
    "submitted_by_representative": false
  },
  "parcel_ref": {
    "parcel_id": {
      "$oid": "6751000000000000000000201"
    },
    "parcel_number": "145",
    "block_number": "12",
    "basin_number": "3",
    "zone_id": "ZONE-RM-01"
  },
  "description": "Ownership transfer application for parcel 145, block 12.",
  "tags": [
    "ownership_transfer",
    "requires_legal_review"
  ],
  "workflow": {
    "current_state": "pre_checked",
    "allowed_next": [
      "survey_required",
      "legal_review",
      "missing_documents",

```

```

    "rejected"
  ],
  "transition_rules_version": "v1.0"
},
"required_documents": [
  {
    "document_type": "ownership_deed",
    "required": true,
    "status": "verified"
  },
  {
    "document_type": "id_copy",
    "required": true,
    "status": "verified"
  },
  {
    "document_type": "sale_contract",
    "required": true,
    "status": "pending_review"
  }
],
"timestamps": {
  "submitted_at": {
    "$date": "2026-02-01T09:00:00Z"
  },
  "pre_checked_at": {
    "$date": "2026-02-01T10:00:00Z"
  },
  "survey_required_at": null,
  "surveyed_at": null,
  "legal_review_at": null,
  "approved_at": null,
  "certificate_issued_at": null,
  "closed_at": null,
  "updated_at": {
    "$date": "2026-02-01T10:00:00Z"
  }
},
"assignment": {
  "assigned_surveyor_id": null,
  "assigned_registrar_id": "staff_14",
  "assignment_policy": "zone+workload+availability"
},
"objection": {
  "has_objection": false,
  "objection_ids": []
},
"internal": {
  "notes": [
    "Initial pre-check completed. Sale contract needs review."
  ],
  "visibility": "staff_only"
}
}

```

Parcels Collection - parcels

```

{
  "_id": {
    "$oid": "67510000000000000000201"
  },

```



```
{
  "_id": {
    "$oid": "67510000000000000000101"
  },
  "full_name": "Nour Ahmad",
  "applicant type": "citizen",
```

```

"identity": {
  "national_id": "400000000",
  "verified": true,
  "verification_method": "otp_stub",
  "verified_at": {
    "$date": "2026-01-20T08:20:00Z"
  }
},
"contacts": {
  "email": "nour@example.com",
  "phone": "+970599000000"
},
"address": {
  "city": "Ramallah",
  "neighborhood": "Al Tireh",
  "zone_id": "ZONE-RM-01"
},
"preferences": {
  "preferred_contact": "email",
  "language": "ar",
  "notifications": {
    "on_status_change": true,
    "on_missing_documents": true,
    "on_certificate_ready": true
  }
},
"stats": {
  "total_applications": 3,
  "approved_applications": 1,
  "pending_applications": 2
},
"created_at": {
  "$date": "2026-01-20T08:10:00Z"
}
}

```

Staff Members Collection - staff_members

```

{
  "_id": {
    "$oid": "67510000000000000000301"
  },
  "staff_code": "SURV-RM-04",
  "name": "Survey Team A",
  "role": "surveyor",
  "department": "Cadastral Survey",
  "skills": [
    "boundary_survey",
    "parcel_subdivision",
    "gps_mapping"
  ],
  "coverage": {
    "zone_ids": [
      "ZONE-RM-01",
      "ZONE-RM-02"
    ],
    "geo_fence": {
      "type": "Polygon",
      "coordinates": [
        [

```

```

        35.19,
        31.89
    ],
    [
        35.22,
        31.89
    ],
    [
        35.22,
        31.92
    ],
    [
        35.19,
        31.92
    ],
    [
        35.19,
        31.89
    ]
]
]
}
},
"schedule": {
    "timezone": "Asia/Jerusalem",
    "shifts": [
        {
            "day": "Mon",
            "start": "08:00",
            "end": "16:00"
        },
        {
            "day": "Tue",
            "start": "08:00",
            "end": "16:00"
        },
        {
            "day": "Wed",
            "start": "08:00",
            "end": "16:00"
        }
    ],
    "on_call": false
},
"workload": {
    "active_tasks": 4,
    "max_tasks": 10
},
"contacts": {
    "phone": "+970599111111",
    "email": "survey_a@example.com"
},
"active": true,
"created_at": {
    "$date": "2026-01-10T09:00:00Z"
}
}

```

Survey Tasks Collection - survey_tasks

```

{
  "_id": {
    "$oid": "67510000000000000000401"
  },
  "task_id": "SURV-2026-0001",
  "application_id": {
    "$oid": "67510000000000000000001"
  },
  "parcel_id": {
    "$oid": "67510000000000000000201"
  },
  "assigned_surveyor_id": {
    "$oid": "67510000000000000000301"
  },
  "status": "visit_scheduled",
  "milestones": [
    {
      "type": "assigned",
      "at": {
        "$date": "2026-02-02T08:00:00Z"
      },
      "by": "system",
      "meta": {
        "reason": "zone and workload match"
      }
    },
    {
      "type": "visit_scheduled",
      "at": {
        "$date": "2026-02-03T09:00:00Z"
      },
      "by": "surveyor",
      "meta": {
        "scheduled_date": "2026-02-05"
      }
    }
  ],
  "field_notes": [],
  "report_uploaded": false,
  "created_at": {
    "$date": "2026-02-02T08:00:00Z"
  }
}

```

Performance Logs Collection - performance_logs

```

{
  "_id": {
    "$oid": "67510000000000000000501"
  },
  "application_id": {
    "$oid": "67510000000000000000001"
  },
  "event_stream": [
    {
      "type": "submitted",
      "by": {
        "actor_type": "applicant",
        "actor_id": "67510000000000000000101"
      }
    },
  ],
}

```

```

    "at": {
      "$date": "2026-02-01T09:00:00Z"
    },
    "meta": {
      "channel": "web"
    }
  },
  {
    "type": "pre_checked",
    "by": {
      "actor_type": "registrar",
      "actor_id": "staff_14"
    },
    "at": {
      "$date": "2026-02-01T10:00:00Z"
    },
    "meta": {
      "missing_documents": 0
    }
  },
  {
    "type": "survey_assigned",
    "by": {
      "actor_type": "system",
      "actor_id": "assignment_engine"
    },
    "at": {
      "$date": "2026-02-02T08:00:00Z"
    },
    "meta": {
      "assigned_surveyor": "SURV-RM-04"
    }
  }
],
"computed_kpis": {
  "processing_days": null,
  "precheck_minutes": 60,
  "survey_delay_days": null,
  "certificate_issued": false
}
}

```

Certificates Collection - certificates

```

{
  "_id": {
    "$oid": "67510000000000000000601"
  },
  "certificate_id": "CERT-2026-0001",
  "application_id": {
    "$oid": "67510000000000000000001"
  },
  "parcel_id": {
    "$oid": "67510000000000000000201"
  },
  "certificate_type": "ownership_certificate",
  "status": "issued",
  "issued_to": {
    "applicant_id": {
      "$oid": "67510000000000000000101"
    }
  },
}

```

```

    "full_name": "Nour Ahmad"
  },
  "issued_at": {
    "$date": "2026-02-10T12:00:00Z"
  },
  "issued_by": "registrar_09",
  "verification": {
    "qr_code_url": "/certificates/CERT-2026-0001/verify",
    "digital_signature_stub": "signed_hash_example"
  }
}

```

Required MongoDB Indexes

```

db.land_applications.create_index("application_id", unique=True)
db.land_applications.create_index("status")
db.land_applications.create_index("application_type")
db.land_applications.create_index("parcel_ref.parcel_number")
db.land_applications.create_index("parcel_ref.zone_id")
db.land_applications.create_index("timestamps.submitted_at")

db.parcels.create_index("parcel_code", unique=True)
db.parcels.create_index("geometry", "2dsphere")
db.parcels.create_index("zone_id")

db.applicants.create_index("identity.national_id", unique=True)
db.staff_members.create_index("staff_code", unique=True)
db.survey_tasks.create_index("application_id")
db.certificates.create_index("certificate_id", unique=True)

```

Annex A - User Interfaces Assignment

Number of Students: 3. Each student is responsible for one complete and functional interface.

Student 1 - Applicant Portal UI

Purpose: Enable citizens, lawyers, companies, and representatives to submit land registration applications, upload documents, and track application status.

Submit Land Application

- Select application type.
- Enter applicant information.
- Enter parcel number, block number, basin number, and zone.
- Choose parcel location on map.
- Upload required documents.
- Submit application.

Application Confirmation

- Application ID.
- Current status.
- Submitted date.
- Required next step.
- Parcel location on map.

Track Application

- Search by application ID.
- View status timeline.
- View missing documents.
- View registrar notes visible to applicant.
- View survey status.

Upload Additional Documents

- Select document type.
- Upload file or register document metadata.
- View document review status.

Submit Objection

- Enter objection reason.
- Attach supporting documents.
- View objection status.

Student 2 - Registrar and Staff Console UI

Purpose: Enable land authority staff to pre-check applications, review documents, manage workflow, and make decisions.

Staff Dashboard

- Total pending applications.
- Applications by status.
- Applications requiring legal review.
- Applications with missing documents.
- Applications under objection.

Application Management

- Searchable and filterable table.
- Filter by status, type, zone, date, applicant, and parcel.
- Open application details.

Application Details

- View applicant details.
- View parcel details.
- View uploaded documents.
- View map.
- Add internal notes.
- Change status.
- Request missing documents.
- Reject application.
- Approve application.

Registrar Review Screen

- Review legal documents.
- Accept or reject documents.
- Add registrar decision.
- Send application to approval or rejection.

Certificate Issuance Screen

- View approved applications.
- Generate certificate metadata.
- Show certificate status.

Student 3 - Surveyor, Map, and Analytics UI

Purpose: Enable surveyors to manage field tasks and enable managers to view spatial and analytical insights.

My Survey Tasks

- Assigned tasks list.
- Parcel number.
- Zone.

- Priority.
- Scheduled visit date.
- Current milestone.

Survey Task Execution

- Mark visit scheduled.
- Mark arrived on site.
- Mark survey started.
- Mark survey completed.
- Upload survey report metadata.
- Add field notes.

Live Parcel Map

- Show parcel boundaries.
- Show pending applications.
- Show survey-required applications.
- Show disputed parcels.
- Use marker clustering.
- Use filters by zone, type, and status.

Analytics Dashboard

- Applications over time.
- Pending applications by zone.
- Average processing time.
- Surveyor workload.
- Applications under objection.
- Certificates issued per month.

Annex B - System Story Workflow

1. Application Submission

An applicant opens the LRMIS portal and submits a land registration application. The applicant enters personal information, selects the application type, enters parcel details, chooses the location on the map, and uploads required documents. The system stores the application with status submitted.

2. Automatic Validation

The system validates required fields, checks the parcel number format, checks whether documents are attached, and records the application in the audit log. If information is missing, the application may be moved to missing_documents.

3. Pre-Check by Staff

A land authority staff member reviews the application. The staff member checks applicant information, parcel information, ownership documents, and application type. If complete, the status becomes pre_checked.

4. Survey Requirement

If the application requires field verification, the system moves the application to survey_required. The system selects a suitable surveyor based on zone, availability, skills, and workload.

5. Field Survey

The surveyor views the assigned task, schedules a visit, arrives on site, performs the survey, and uploads survey report metadata. The application status becomes surveyed.

6. Legal Review

The registrar reviews the ownership documents, survey report, parcel information, and any objections. The application status becomes legal_review.

7. Objection Handling

If another party submits an objection, the application moves to `under_objection`. The registrar reviews the objection and decides whether to continue, reject, or hold the application.

8. Approval

If all requirements are satisfied, the registrar approves the application. The application status becomes approved.

9. Certificate Issuance

The system generates certificate metadata, including certificate ID, issue date, parcel reference, applicant reference, and verification QR-code stub. The application status becomes `certificate_issued`.

10. Closure

After the certificate is issued and all records are updated, the application is closed. The final status becomes closed.

11. Mapping and Analytics

Pending applications, disputed parcels, survey tasks, and registered parcels appear on the live map. Resolved and closed applications contribute to analytics and management reports.

Suggested Minimum Pages / Screens

35. Login or simple user selection page.
36. Applicant dashboard.
37. Submit application page.
38. Track application page.
39. Staff dashboard.
40. Application management table.
41. Application details page.
42. Surveyor task list.
43. Survey task execution page.
44. Live map page.
45. Analytics dashboard.
46. Certificate view page.

Suggested Technologies

Backend

- FastAPI
- PyMongo
- Pydantic
- Uvicorn
- Python-dotenv
- Passlib or simple token-based authentication

Database

- MongoDB
- MongoDB Compass
- MongoDB Atlas or local MongoDB

Frontend

- React
- Vue.js
- Angular
- Streamlit

Map

- OpenStreetMap
- Leaflet
- GeoJSON

Documentation

- Swagger/OpenAPI
- Postman
- README.md

Final Notes

The project should not be a simple CRUD system. It must demonstrate workflow rules, validation, geospatial data, assignment logic, audit logs, dashboards, realistic land registration operations, clean API design, clear database design, functional UI screens, and the ability to explain and modify the code during discussion.